UK8S KRO Stack

GitHub-First Kubernetes Infrastructure with Kyverno Policy Management

Declarative Infrastructure	Policy-Driven Security	GitOps Automation
KRO + Azure Service Operator	Kyverno Admission Control	Flux CD Continuous Delivery

Azure Kubernetes Service | Management Groups | Azure AD Integration

Agenda

1. **Executive Summary**
   Why GitHub-First Infrastructure?
2. **Architecture Overview**
   Three-Layer KRO Stack Design
3. **GitOps Workflow**
   Infrastructure as Code Pipeline
4. **Kyverno Policy Management**
   Security & Compliance Governance
5. **Management Cluster Strategy**
   Per Management Group Architecture
6. **Azure AD Integration**
   Identity & Access Management

7. Certification & Validation

Automated Compliance Checks

8. Benefits & Value

Business Value Summary

Executive Summary

Why GitHub-First Infrastructure?

Traditional Approach

- ✗ Manual cluster provisioning
- ✗ Configuration drift over time
- ✗ Inconsistent security policies
- ✗ No audit trail for changes
- ✗ Slow disaster recovery

GitHub-First Approach

- ✓ Declarative cluster definitions
- ✓ Git as single source of truth
- ✓ Policy-as-Code enforcement
- ✓ Complete audit history
- ✓ Instant DR from Git state

50 UAMIs → 5 UAMIs	10x Faster Recovery	100% Policy Compliance
Shared Identity Model	GitOps-Based DR	Automated Enforcement

Three-Layer Architecture

KRO Stack Design: Deploy Once, Share Everywhere

Layer 1: Platform Foundation

Deploy ONCE - Shared Resources

- 5 Shared User-Assigned Managed Identities (UAMIs)
- External Secrets, External DNS, Cert Manager, Grafana, Flux
- Central Resource Group for platform-wide resources
- Single RBAC assignment point for all clusters

Layer 2: Management Cluster

Deploy ONCE per Management Group

- AKS Management Cluster with KRO, ASO, Flux installed
- Federated Identity Credentials linked to shared UAMIs
- Service Accounts for workload identity
- Platform controllers (cert-manager, external-dns, ESO)

Layer 3: Worker Clusters

Deploy PER CLUSTER - Unlimited Scale

- AKS Worker Clusters for application workloads
- References SAME shared UAMIs from Layer 1
- Federated Credentials bind UAMIs to cluster OIDC
- Simple YAML to add new clusters in minutes

GitOps Workflow

Infrastructure as Code Pipeline

1	**Developer**	Commits cluster YAML to Git repository
2	**Pull Request**	Code review + policy validation gates
3	**Merge to Main**	Triggers Flux CD sync on management cluster
4	**KRO Controller**	Reconciles ResourceGraphDefinition
5	**ASO Controller**	Provisions Azure resources (AKS, UAMIs, etc.)
6	**Certification**	Argo Workflow validates deployment

Key Benefit: Every infrastructure change is versioned, reviewed, and auditable. Disaster recovery is as simple as re-syncing from Git.

Kyverno Policy Management

Security & Compliance Governance

Admission Control	• Validates all Kubernetes resources before creation • Enforces naming conventions and labels • Blocks non-compliant configurations
Security Policies	• Pod Security Standards enforcement • Network policy requirements • Image registry restrictions
Resource Governance	• Resource quota enforcement • Namespace isolation rules • Storage class restrictions
Audit & Compliance	• Real-time policy violation alerts • Compliance reporting dashboards • Drift detection and remediation

Integration: Kyverno policies are stored in Git and deployed via Flux, ensuring policy-as-code consistency across all management groups.

Management Cluster Strategy

One Management Cluster per Management Group

Management Group: Production

Management Cluster (1)
- KRO Controller
- Azure Service Operator
- Flux CD + Kyverno Policies

Worker Clusters (N)
- prod-app-cluster-001
- prod-app-cluster-002
- prod-data-cluster-001

Management Group: Non-Production

Management Cluster (1)
- KRO Controller
- Azure Service Operator
- Flux CD + Kyverno Policies

Worker Clusters (N)
- dev-test-cluster-001
- staging-cluster-001
- qa-cluster-001

Blast Radius Isolation: Issues in non-prod don't affect production

Policy Separation: Different Kyverno policies per environment tier

RBAC Boundaries: Azure AD groups scoped to management groups

Cost Attribution: Clear subscription/cost center mapping

Azure AD Integration

Identity & Access Management

Component	Identity Type	Purpose
AKS Control Plane	User-Assigned MI	Cluster operations, Azure API access
Kubelet	User-Assigned MI	Node operations, ACR image pull
External Secrets	Workload Identity	Key Vault secret access
External DNS	Workload Identity	DNS zone record management
Cert Manager	Workload Identity	Certificate issuance
Flux Controllers	Workload Identity	Git repo + ACR access

Azure AD Group-Based Access

AKS Cluster Admins: Full cluster admin access via Azure RBAC

Platform Engineers: KRO/ASO management, cluster provisioning

Application Teams: Namespace-scoped access, workload deployment

Security/Audit: Read-only access for compliance monitoring

Automated Certification

Argo Workflows Validation Pipeline

1	**Configuration**	Validates cluster YAML and parameters
2	**KRO Resources**	Checks ResourceGraphDefinitions are Active
3	**ASO Resources**	Verifies Azure resources are Ready
4	**Flux GitOps**	Confirms Flux controllers are running
5	**Connectivity**	Tests API server and node health
6	**Security**	Validates workload identity and policies

Total Duration	~70 seconds
Trigger	Automatic on deployment + Weekly scheduled
Output	JSON certification report + Argo UI dashboard

Benefits & Value

Business Value Summary

90%	Reduction in identity management overhead
10x	Faster disaster recovery time
100%	Policy compliance enforcement
~70s	Automated certification per cluster

Key Benefits

- **Operational Efficiency:** Single management cluster manages unlimited worker clusters

- **Security Posture:** Kyverno policies enforced at admission, preventing drift
- **Developer Experience:** Simple YAML to provision complete AKS clusters
- **Audit & Compliance:** Complete Git history + automated certification
- **Cost Optimization:** Shared identities reduce Azure AD object sprawl

Next Steps

Phase 1	**Pilot Deployment** Deploy platform foundation + 1 management cluster in non-prod
Phase 2	**Policy Rollout** Configure Kyverno policies for security baseline
Phase 3	**Production Ready** Deploy production management group infrastructure
Phase 4	**Scale Out** Add worker clusters as needed via GitOps

Resources: GitHub Repository | Architecture Docs | Runbooks

Questions?